Ch11 標準函式庫

Ch11 標準函式庫

- Python 提供許多好用的模組，可以直接匯入使用，節省使用者開發程式的時間，在寫程式過程中可以先找一下標準函式庫是否已經有類似功能的程式，可以節省撰寫程式的時間，而這樣的哲學，Python 稱作內建電池(batteries included)，以下介紹常用的電池。

11-1 系統相關的模組

- 模組os 的其他重要函式

模組 os 重要函式與說明	程式碼與執行結果
os.system(command) 讓系統執行 command 所指定的指令，Windows 作業系統中指令是任何可以在「命令提示字元」執行的程式，Linux 作業系統中指令是任何可以在「Shell」執行的程式。	os.system('dir') os.system('ls') 註：Windows 作業系統使用「dir」，Linux 作業系統使用「ls」。
	列出目前所在目錄的檔案與資料夾。
os.getenv(key) 查詢系統 key 所對應的值。	print(os.getenv('USERNAME'))
	user

11-1　系統相關的模組

- 若要顯示程式目前的區域變數與全域變數，需使用以下系統函式，函式locals 顯示區域變數，而函式globals 顯示全域變數。

模組 os 重要函式與說明	程式碼與執行結果
locals() 系統使用字典儲存區域變數。	print(locals())
	{'__doc__': None, '__builtins__': <module 'builtins' (built-in)> , 省略一部分資料 ,'__name__': '__main__'}
globals() 系統使用字典儲存全域變數。	print(globals())
	{'__doc__': None, '__builtins__': <module 'builtins' (built-in)> , 省略一部分資料 ,'__name__': '__main__'}

模組pprint 的重要函式

模組 pprint 重要函式與說明	程式碼與執行結果
pprint.pprint(data) 將 data 顯示在螢幕上。	pprint.pprint(globals()) {'__builtins__': <module 'builtins' (built-in)>, 省略一部分資料 , '__name__': '__main__', 省略一部分資料 , 'pprint': <module 'pprint' from 'C:\\Users\\ user\\AppData\\Local\\Programs\\Python\\ Python35-32\\lib\\pprint.py'>}

11-1-1　執行指令與顯示環境變數

- (ch11\11-1-1- 執行指令與顯示環境變數.py)
- 使用「os.system」執行指令，接著利用「os.getenv」讀取系統環境變數。

行號	範例 ( ：ch11\11-1-1- 執行指令與顯示環境變數 .py)	執行結果
1 2 3 4 5 6	import os os.system('dir') print(os.getenv('COMPUTERNAME')) print(os.getenv('HOMEDRIVE')) print(os.getenv('HOMEPATH')) print(os.getenv('USERNAME'))	顯示程式所在資料夾下的檔案與資料夾，資料過多，省略執行結果。 USER-PC C: \Users\user user 註：每個電腦的電腦名稱、家目錄所在資料夾與使用者名稱不一定相同。

程式解說

- 第1 行：匯入模組os。
- 第2 行：使用模組os 的函式system 執行指令「dir」，會顯示程式所在資料夾的檔案與資料夾。
- 第3 行：使用函式print 顯示模組os 的函式getenv('COMPUTERNAME')，顯示電腦名稱到螢幕上。
- 第4 行：使用函式print 顯示模組os 的函式getenv('HOMEDRIVE')，顯示家目錄所在磁碟機到螢幕上。

- 第5 行：使用函式print 顯示模組os 的函式getenv('HOMEPATH')，顯示家目錄所在路徑到螢幕上。
- 第6 行：使用函式print 顯示模組os 的函式getenv('USERNAME')，顯示使用者名稱到螢幕上。

11-1-2　使用pprint 顯示區域變數與全域變數

- (ch11\11-1-2- 顯示區域變數與全域變數.py)
- 使用「print」與「pprint.pprint」分別顯示區域變數與全域變數，比較兩者的差異。

行號	範例 ( ：ch11\11-1-2- 顯示區域變數與全域變數 .py)
1	import pprint
2	print(' 在函式外，顯示全域變數 ', globals())
3	print(' 在函式外，顯示區域變數 ', locals())
4	def add(x, y):
5	sum = x+y
6	print(' 在函式內，顯示全域變數 ')
7	pprint.pprint(globals())
8	print(' 在函式內，顯示區域變數 ')
9	pprint.pprint(locals())
10	return sum
11	ans = add(1,2)

程式解說

- **第1 行**：匯入模組pprint。
- **第2 行**：顯示字串「在函式外，顯示全域變數」與全域變數到螢幕。
- **第3 行**：顯示字串「在函式外，顯示區域變數」與區域變數到螢幕。

- 第4 到10 行：定義函式add，輸入兩個參數x 與y，設定sum 為x 加y(第5 行)，顯示字串「在函式內，顯示全域變數」(第6 行)，呼叫模組pprint 的函式pprint 顯示全域變數到螢幕(第7 行)
- 顯示字串「在函式內，顯示區域變數」(第8 行)，呼叫模組pprint 的函式pprint 顯示區域變數到螢幕(第9 行)，回傳變數sum(第10行)。

- 第11 行：呼叫函式add，以1 與2 為參數，將結果指定給變數ans。

11-1-2 使用pprint 顯示區域變數與全域變數

執行結果

```
在函式外，顯示全域變數 {'__spec__': None, '__cached__': None, 省略一部分資料 , '__
builtins__': <module 'builtins' (built-in)>}
在函式外，顯示區域變數 {'__spec__': None, '__cached__': None, 省略一部分資料 , '__
builtins__': <module 'builtins' (built-in)>}
在函式內，顯示全域變數
{'__builtins__': <module 'builtins' (built-in)>,
 '__cached__': None, 省略一部分資料 ,
 '__spec__': None,
 'add': <function add at 0x009E6C90>,
 'pprint': <module 'pprint' from 'C:\\Users\\pr3\\AppData\\Local\\Programs\\Python\\
Python35-32\\lib\\pprint.py'>}
在函式內，顯示區域變數
{'sum': 3, 'x': 1, 'y': 2}
```

11-2 可迭代的函式庫

- Python 提供許多可以處理可迭代物件的模組和函式，包含模組itertools、函式enumerate、zip、filter、map 與reduce 等，以下分別介紹。

11-2-1　模組itertools

- 模組itertools 只要給定起始數字與遞增(減) 值就可以產生無窮數列，也可以利用循環與重複功能產生數列，可以對數列進行運算，與產生指定元素的排列組合結果。

模組 itertools 重要函式與說明	程式碼與執行結果
itertools.count(start,[step]) 產生資料列，起始數字爲 start，每次遞增 step。	import itertools nums = itertools.count(1,2)
	nums 爲無窮數列 1 3 5 7 …
itertools.cycle(iterable) 使用 iterable 不斷循環產生資料列。	import itertools nums = [1, 2, 3] cyclenums = itertools.cycle(nums)
	cyclenums 爲無窮數列 1 2 3 1 2 3 1 2 3 …

11-2-1 模組itertools

模組 itertools 重要函式與說明	程式碼與執行結果
itertools.repeat(object[,times]) 使用 object 重複 times 次產生資料列。	import itertools nums = [1, 2, 3] repeatnums = itertools.repeat(nums, 2) print(list(repeatnums))
	[[1, 2, 3], [1, 2, 3]]
itertools.accumulate(iterable[,func]) 將 iterable 物件使用函式 func 進行運算。	import itertools nums = [i for i in range(1, 6)] sums = itertools.accumulate(nums) print(list(sums))
	[1, 3, 6, 10, 15]
itertools.chain(*iterables) 將多個 iterable 物件，依序取出每個 iterable 的所有元素，串接成新的數列	import itertools s = itertools.chain('Py','thon') print(list(s))
	['P', 'y', 't', 'h', 'o', 'n']

11-2-1　模組itertools

itertools.permutations(iterable, r=None) 從 iterable 取 r 個元素的所有排列，若 r 爲 None，則排列所有元素。	import itertools perm = itertools.permutations('ABC', r=2) print(list(perm))
	[('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
itertools.combinations(iterable, r) 從 iterable 取 r 個元素的所有組合。	import itertools comb = itertools.combinations('ABC', 2) print(list(comb))
	[('A', 'B'), ('A', 'C'), ('B', 'C')]

11-2-1　模組itertools

- 1. 模組itertools 的函式count 範例

行號	範例 ( ：ch11\11-2-1a- 模組 itertools 的函式 count.py)	執行結果
1	import itertools	1
2	nums = itertools.count(1,2)	3
3	for i in range(5):	5
4	num = nums.__next__()	7
5	print(num)	9

程式解說

- 第1 行：匯入模組itertools。
- 第2 行：使用模組itertools的函式count(1,2)，將此物件指定給變數nums。
- 第3到5行：使用for迴圈與range(5)讓迴圈執行五次，迴圈內使用「nums.__next__()」取出nums 的下一個元素指定給變數num(第4 行)，印出變數num 到螢幕(第5 行)。

11-2-1　模組itertools

- 2. 模組itertools 的函式cycle 範例

行號	範例 ( ：ch11\11-2-1b- 模組 itertools 的函式 cycle.py)	執行結果
1	import itertools	1
2	nums = [1, 2, 3]	2
3	cyclenums = itertools.cycle(nums)	3
4	for i in range(6):	1
5	num = cyclenums.__next__()	2
6	print(num)	3

程式解說

- 第1 行：匯入模組itertools。
- 第2 行：串列nums設定為「1,2,3」
- 第3 行：使用模組itertools 的函式cycle，以串列nums 為輸入，以串列nums 重複循環產生數列，將此物件指定給變數cyclenums。

- 第4 到6 行：使用for 迴圈與range(6) 讓迴圈執行六次，迴圈內使用「cyclenums.__next__()」取出cyclenums的下一個元素指定給變數num(第5 行)，印出變數num到螢幕(第6 行)。

11-2-1　模組itertools

- 3. 模組itertools 的函式repeat 範例

行號	範例 ( ：ch11\11-2-1c- 模組 itertools 的函式 repeat.py)	執行結果
1	import itertools	
2	nums = [1, 2, 3]	
3	repeatnums = itertools.repeat(nums, 3)	[1, 2, 3]
4	for i in range(3):	[1, 2, 3]
5	num = repeatnums.__next__()	[1, 2, 3]
6	print(num)	

程式解說

- 第1 行：匯入模組itertools。
- 第2 行：串列nums設定為「1,2,3」
- 第3 行：使用模組itertools 的函式repeat，以串列nums 與數字3 為輸入，以串列nums 重複3 次產生物件，將此物件指定給變數repeatnums。

- 第4到6行：使用for迴圈與range(3)讓迴圈執行三次，迴圈內使用「repeatnums.__next__()」取出repeatnums 的下一個元素指定給變數num(第5 行)，印出變數num到螢幕(第6行)。

11-2-1　模組itertools

- 4. 模組itertools 的函式accumulate 範例

行號	範例 ( ：ch11\11-2-1d- 模組 itertools 的函式 accumulate.py)	執行結果
1	import itertools	[1, 3, 6, 10, 15] [1, 2, 6, 24, 120]
2	nums = [i for i in range(1, 6)]	
3	sums = itertools.accumulate(nums)	
4	print(list(sums))	
5	sums = itertools.accumulate(nums, lambda x,y:x*y)	
6	print(list(sums))	

程式解說

- 第1 行：匯入模組itertools。
- 第2 行：使用串列產生式，產生串列「1,2,3,4,5」指定給變數nums。
- 第3 行：使用模組itertools 的函式accumulate，以串列nums為輸入產生物件，將此物件指定給變數sums。

- 第4行：使用函式list將變數sums轉換成串列，使用函式print將串列顯示在螢幕。
- 第5 行：使用模組itertools 的函式accumulate，以串列nums 與兩個數字相乘的lambda 函式為輸入產生物件，將此物件指定給變數sums。
- 第6行：使用函式list將變數sums轉換成串列，使用函式print將串列顯示在螢幕。

- 5. 模組itertools 的函式chain 範例

行號	範例 ( ：ch11\11-2-1e- 模組 itertools 的函式 chain.py)	執行結果
1	import itertools	
2	s = itertools.chain('Py','thon')	['P', 'y', 't', 'h', 'o', 'n']
3	print(list(s))	

程式解說

- 第1 行：匯入模組itertools。
- 第2 行：使用模組itertools 的函式chain，以字串「Py」與字串「thon」為輸入產生物件，將此物件指定給變數s。
- 第3 行：使用函式list將變數s轉換成串列，使用函式print 將串列顯示在螢幕。

- 6. 模組itertools 的函式permutations 與 combinations 範例

行號	範例 ( : ch11\11-2-1f- 模組 itertools 的函式 permutations 與 combinations.py)	執行結果
1	import itertools	
2	perm = itertools.permutations('ABC', 2)	[('A', 'B'), ('A', 'C'), ('B', 'A'),
3	print(list(perm))	('B', 'C'), ('C', 'A'), ('C', 'B')]
4	comb = itertools.combinations('ABC', 2)	[('A', 'B'), ('A', 'C'), ('B', 'C')]
5	print(list(comb))	

程式解說

- 第1 行：匯入模組itertools。
- 第2 行：使用模組itertools 的函式permutations，以字串「ABC」與數字2為輸入產生從字串「ABC」中挑選2 個字元的所有排列可能性，將此排列物件指定給變數perm。

- 第3行：使用函式list將變數perm轉換成串列，使用函式print將串列顯示在螢幕。
- 第4 行：使用模組itertools的函式combinations，以字串「ABC」與數字2為輸入產生從字串「ABC」中挑選2 個字元的所有組合可能性，將此物件指定給變數comb。
- 第5行：使用函式list將變數comb轉換成串列，使用函式print將串列顯示在螢幕。

11-2-2 enumerate 與zip

- 函式enumerate 將可迭代物件的每一個元素給予編號，函式zip 可以自動配對兩個以上可迭代物件的每個元素，一個一個依序對應產生新的物件。

函式與說明	程式碼與執行結果
enumerate(sequence, start=0) 將 sequence 中元素進行編號，由 start 開始編號。	days = [' 星期天 ', ' 星期一 ', ' 星期二 ', ' 星期三 ', ' 星期四 ', ' 星期五 ', ' 星期六 '] p = enumerate(days, start=1) print(list(p))
	[(1, ' 星期天 '), (2, ' 星期一 '), (3, ' 星期二 '), (4, ' 星期三 '), (5, ' 星期四 '), (6, ' 星期五 '), (7, ' 星期六 ')]
zip([iterable, ...]) 將所輸入的多個 iterable，每個 iterable 中依序由前到後每次取一個組成一個 tuple，這些 tuple 組成 zip 物件。	a = [1,2,3] b = ['a','b','c'] c = zip(a,b) print(list(c))
	[(1, 'a'), (2, 'b'), (3, 'c')]

以下為enumerate 與zip 範例。

行號	範例 (: ch11\11-2-2-enumerate 與 zip.py)	執行結果
1	days = ['星期天', '星期一', '星期二', '星期三', '星期四', '星期五', '星期六']	1 星期天 2 星期一
2	p = enumerate(days, start=1)	3 星期二
3	for c, day in p:	4 星期三
4	print(c, day)	5 星期四
5	do = ['休息', '游泳','跑步', '籃球', '桌球', '羽球', '棒球', '壘球']	6 星期五
6	week = zip(days, do)	7 星期六
7	for day,sport in week:	星期天 休息
8	print(day,sport)	星期一 游泳 星期二 跑步 星期三 籃球 星期四 桌球 星期五 羽球 星期六 棒球

程式解說

- 第1 行： 設定days為串列「' 星期天', ' 星期一', ' 星期二', ' 星期三', ' 星期四', ' 星期五', ' 星期六' 」。
- 第2 行：使用函式enumerate 以串列days 為輸入，start 為1，將串列days 由1 開始編號，將獲得的enumerate 物件指定給變數p。

- 第3 到4 行：使用for 迴圈與tuple 開箱(unpacking)，將enumerate 物件的變數p
- 取出每一個元素，編號儲存到變數c，元素儲存到變數day，顯示變數c 與變數day到螢幕。

- 第5行： 設定do為串列「'休息', '游泳','跑步', '籃球', '桌球', '羽球', '棒球', '壘球'」。
- 第6 行：使用函式zip 以串列days與串列do為輸入，將獲得的zip物件指定給變數week，串列do 元素較串列days 多一個元素，所以串列do 最後一個元素「壘球」不會使用到。

- 第7 到8 行：使用for 迴圈依序取出變數week 的每一個元素，第一個元素儲存到變數day，第二個元素儲存到變數sport，顯示變數day 與變數sport 到螢幕上。

11-2-3 filter、map 與reduce

- 函式filter 過濾可迭代物件，只保留符合條件的元素
- 函式map 將多個可迭代物件根據輸入map的函式進行運算，回傳一個可迭代物件

- 模組functools 的函式reduce將輸入的可迭代物件由左到右，使用輸入reduce 的函式每次取可迭代物件的最左邊兩個進行運算，將運算結果與第3 個物件進行運算
- 依此類推直到輸入的可迭代物件的所有元素都計算過，回傳計算的結果。

函式與說明	程式碼與執行結果
filter (function, iterable) 使用 function 作用到 iterable 的每個元素，將結果爲 True 的元素保留下來。	nums = [i for i in range(1,10)] nums2 = filter(lambda x:x%2, nums) print(list(nums2))
	[1, 3, 5, 7, 9]

函式與說明	程式碼與執行結果
map(function, iterable, ...) 使用 function 作用到一個以上 iterable 的每個元素，function 若接收兩個參數，iterable 就需要兩個，最後回傳 function 作用到 iterable 的結果。	nums = [i for i in range(1,6)] def double(x): return 2*x nums2 = map(double, nums) print(list(nums2)) [2, 4, 6, 8, 10]

函式與說明	程式碼與執行結果
functools.reduce(function, iterable[, initializer]) function 需要能輸入兩個參數，使用 function 作用到 iterable 由左到右的每個元素。如果有 initializer，則計算結果的初始值設定爲 initializer，將第 1 個元素與第 2 個元素輸入到 function，產生的數字與第 3 個元素輸入到 function，產生的數字與第 4 個元素輸入到 function，依此類推直到輸入的可迭代物件的所有元素都計算過，回傳此計算結果。	from functools import reduce num = reduce(lambda x, y:x+y, range(1,10)) print(num) 45

1. 函式filter 範例

行號	範例 ( ：ch11\11-2-3a-filter.py)	執行結果
1	nums = [i for i in range(1,10)]	[1, 3, 5, 7, 9] [2, 4, 6, 8]
2	nums2 = filter(lambda x:x%2, nums)	
3	print(list(nums2))	
4	nums2 = filter(lambda x:x%2 == 0, nums)	
5	print(list(nums2))	

程式解說

- **第1 行：**使用串列生成式，使用range(1, 10) 產生數字1 到9 的串列指定給變數nums。

程式解說

- **第2行**：使用函式filter將串列nums的每一個元素除以2的餘數為1的元素保留下來，將保留下來的元素指定給變數nums2。
- **第3 行**：將變數nums2使用函式list轉換成串列，最後使用函式print 印出此串列。

程式解說

- **第4行**：使用函式filter將串列nums的每一個元素除以2的餘數為0的元素保留下來，將保留下來的元素指定給變數nums2。
- **第5 行**：將變數nums2使用函式list轉換成串列，最後使用函式print 印出此串列。

2. 函式map 範例

行號	範例 ( ：ch11\11-2-3b-map.py)	執行結果
1	nums = [i for i in range(1,6)]	
2	def double(x):	
3	return 2*x	
4	nums2 = map(double, nums)	
5	print(list(nums2))	[2, 4, 6, 8, 10]
6	a = [i for i in range(1,6)]	[6, 14, 24, 36, 50]
7	b = [i for i in range(6,11)]	
8	c = map(lambda x,y:x*y, a, b)	
9	print(list(c))	

程式解說

- **第1 行**：使用串列生成式，使用range(1, 6)產生數字1 到5 的串列指定給變數nums。
- **第2 到3 行**：定義函式double，輸入參數x，回傳2 乘以x。
- **第4 行**：使用函式map 與函式double 將串列nums 的每一個元素乘以2，將乘以2的map 物件指定給變數nums2。

- **第5 行**：將變數nums2使用函式list轉換成串列，最後使用函式print 印出此串列。
- **第6 行**：使用串列生成式，使用range(1, 6)產生數字1 到5 的串列指定給變數a。

- **第7行**：使用串列生成式，使用range(6, 11)產生數字6到10的串列指定給變數b。
- **第8 行**：使用函式map 與lambda 函式將串列a 與串列b 的每一個元素依序相乘，將相乘後的map 物件指定給變數c。
- **第9 行**：將變數c 使用函式list轉換成串列，最後使用函式print 印出此串列。

3. 模組functools 的函式reduce 範例

行號	範例 ( ：ch11\11-2-3c-reduce.py)	執行結果
1	import functools	
2	num = functools.reduce(lambda x, y:x+y, range(1,10))	45
3	print(num)	48
4	num = functools.reduce(lambda x, y:x+y, range(1,10),3)	
5	print(num)	

程式解說

- **第1 行**：匯入模組functools。
- **第2 行**：函式lambda 輸入參數x 與y，回傳x+y 的結果，使用函式functools.reduce與函式lambda 作用於range(1,10)，將計算結果指定到變數num。

- **第3 行**：最後使用函式print 印出變數num。
- **第4 行**：函式lambda 輸入參數x 與y，回傳x+y 的結果，使用函式functools.reduce與函式lambda 作用於range(1,10)，初始值設定為3，將計算結果指定到變數num。
- **第5 行**：最後使用函式print 印出變數num。

11-2-4　篩選法求質數範例

行號	範例 (：ch11\11-2-4-prime.py)	執行結果
1	import itertools	2 3 5 7 11 13 17 19 23 29 31 37 … 967 971 977 983 991 997
2	def iter_primes():	
3	numbers = itertools.count(2)	
4	while True:	
5	prime = numbers.__next__()	
6	yield prime	
7	numbers = filter(prime.__rmod__, numbers)	
8	for p in iter_primes():	
9	if p > 1000:	
10	break	
11	print(p,' ',end='')	

程式解說

- **第1 行**：匯入模組itertools。
- **第2 到7 行**：定義生成式函式iter_primes，設定numbers 為模組itertools 的函式count，由2 開始的無窮數列(第3 行)。

- 使用while 的無窮迴圈，設定prime 為numbers 的下一個數字(第5 行)
- 使用yield 回傳prime，yield 會回到上一次的狀態繼續執行(第6 行)
- 使用函式filter 去掉numbers 中prime 的倍數(第7 行)。

- **第8 到11 行**：使用for 迴圈依序取出生成式iter_primes 到變數p，若變數p 大於
- 1000，則中斷迴圈(第9 到10 行)。使用函式print 顯示變數p 串接空白字元(space)，
- 設定end 為空字元，也就是取消換行(第11 行)。

11-3 時間函式庫

- 時間函式庫包含模組datetime、date 與 time 等，用於顯示目前時間與計算經過的時間

11-3-1 模組datetime 與date

重要函式與說明	程式碼與執行結果
datetime.now() 回傳目前的日期與時間。	from datetime import datetime now = datetime.now() print(now)
	2018-05-04 10:58:50.798852
datetime.datetime(year,month, day, hour=0, minute=0, second=0, microsecond=0) 使用 year、month、day、hour、minute、second、microsecond 建立 datetime 物件。	from datetime import datetime b = datetime(1995,1,1,21,30,0,0) print(b)
	1995-01-01 21:30:00

重要函式與說明	程式碼與執行結果
date.today() 回傳目前的日期。	from datetime import date now = date.today() print(now)
	2018-05-04

行號	範例 (：ch11\11-3-1-datetime 與 date.py)	執行結果
1	from datetime import datetime, date	
2	now = date.today()	2018-05-04
3	print(now)	2018-05-04
4	now = datetime.now()	10:58:50.798852
5	print(now)	2018 5 4
6	print(now.year, now.month, now.day)	10 58 50 798852
7	print(now.hour, now.minute, now.second, now.microsecond)	

程式解說

- **第1 行**：從套件datetime匯入模組datetime與date。
- **第2 行**：使用模組date 的函式today，將此物件指定給變數now。
- **第3 行**：使用函式print 顯示變數now到螢幕上。
- **第4 行**：使用模組datetime的函式now，將此物件指定給變數now。

- **第5 行**：使用函式print 顯示變數now到螢幕上。
- **第6 行**：使用函式print 顯示變數now 的year(目前為西元幾年)、變數now 的month(月份) 與變數now 的day(日) 到螢幕上。

- **第7 行**：使用函式print 顯示變數now的hour(小時)、變數now的minute(分鐘)、變數now 的second(秒) 與變數now 的microsecond(微秒) 到螢幕上。

11-3-2 模組time

重要函式與說明	程式碼與執行結果
time.time() 累計從 1970 年 1 月 1 日凌晨 0 點 0 分 0 秒到目前爲止的秒數。	import time now = time.time() print(now)
	1525403162.0861557
time.ctime([secs]) 將累計秒數 secs 轉換爲以「年月日時分秒」表示的時間，若沒有輸入 secs，則預設以呼叫 time.time() 的回傳值爲輸入。	import time print(time.ctime())
	Fri May 4 11:06:02 2018

重要函式與說明	程式碼與執行結果
time.localtime([secs]) 將累計秒數 secs 轉換爲以物件 struct_time 表示的時間，若沒有輸入 secs，則以呼叫 time.time() 的回傳值爲輸入。	import time t=time.localtime() print(t) time.struct_time(tm_year=2018, tm_mon=5, tm_mday=4, tm_hour=11, tm_min=6, tm_sec=2, tm_wday=4, tm_yday=124, tm_isdst=0)
time.strftime(format[,t]) t 爲 struct_time 物件，以 format 格式顯示出來，若沒有輸入 t，則以呼叫 time.localtime() 的回傳值爲輸入。	import time fmt = "%Y-%m-%d(%a) %H %M %S" print(' 現在時間爲 ',time.strftime(fmt)) 現在時間爲 2018-05-04(Fri) 11 06 02

函式time.strftime 的參數format 的格式

格式	顯示資料	格式	顯示資料
%a	縮寫的星期。	%d	以 01 到 31 顯示日期。
%A	不縮寫的星期。	%j	以 001 到 366 顯示一年的第幾天。
%y	顯示西元最後兩位。	%H	以 00 到 23(24 小時制) 顯示小時。
%Y	顯示西元的四位數。	%I	以 01 到 12(12 小時制) 顯示小時。
%m	以 01 到 12 顯示月份。	%p	顯示 AM 或 PM。
%b	縮寫的月份。	%M	以 00 到 59 顯示分鐘。
%B	不縮寫的月份。	%S	以 00 到 61 顯示秒。

行號	範例 ( ：ch11\11-3-2- 模組 time.py)	執行結果
1	import time	1525403162.0861557 Fri May 4 11:06:02 2018 time.struct_time(tm_year=2018, tm_mon=5, tm_mday=4, tm_hour=11, tm_min=6, tm_sec=2, tm_wday=4, tm_yday=124, tm_isdst=0) 現在時間爲 2018-05-04(Fri) 11 06 02 現在時間爲 2018-May-04(Friday) AM 11 06 02
2	now = time.time()	
3	print(now)	
4	print(time.ctime())	
5	t=time.localtime()	
6	print(t)	
7	fmt = "%Y-%m-%d(%a) %H %M %S"	
8	print(' 現在時間爲 ',time.strftime(fmt))	
9	fmt2 = "%Y-%B-%d(%A) %p %I %M %S"	
10	print(' 現在時間爲 ',time.strftime(fmt2))	

程式解說

- **第1 行**：匯入模組time。
- **第2 行**：使用模組time的函式time，將此物件指定給變數now。
- **第3 行**：使用函式print 顯示變數now到螢幕上。
- **第4 行**：使用函式print 將模組time函式ctime的結果顯示在螢幕上。

- **第5 行**：使用模組time的函式localtime，將此物件指定給變數t。
- **第6 行**：使用函式print 顯示變數t 到螢幕上。
- **第7 行**：設定時間格式字串fmt 為「%Y-%m-%d(%a) %H %M %S」。

- **第8 行**：使用函式print 顯示「目前時間為」，串接模組time函式strftime以時間格式字串fmt 為輸入的結果顯示到螢幕上。

- **第9 行**：設定時間格式字串fmt2 為「%Y-%B-%d(%A) %p %I %M %S」。
- **第10 行**：使用函式print 顯示「目前時間為」，串接模組time 函式strftime 以時間格式字串fmt2 為輸入的結果顯示到螢幕上。

11-3-3 模組timedelta

重要函式與說明	程式碼與執行結果
datetime.timedelta(days=0, seconds=0, microseconds=0, milliseconds=0, minutes=0, hours=0, weeks=0) 表示兩個時間點的差距，以 days、seconds、microseconds、milliseconds、minutes、hours 與 weeks 初始化物件 timedelta。輸入 days 的數值需在 -999999999 到 999999999 之間 (含邊界值)，輸入 seconds 的數值需在 0 到 86399 之間 (含邊界值)，輸入 microseconds 的數值需在 0 到 999999 之間 (含邊界值)。	from datetime import datetime,timedelta now = datetime.now() now1000 = now + timedelta(days=1000) print(now) print(now1000) 2016-06-20 10:28:11.313910 2019-03-17 10:28:11.313910

- 以下為模組time 範例，假設生日為西元1995 年1 月1 日晚上9 點30 分，請計算10000 天後的日期與時間。

行號	範例 ( ：ch11\11-3-3- 模組 timedelta.py)	執行結果
1	from datetime import datetime, timedelta	
2	birthday = datetime(1995,1,1,21,30,0,0)	
3	print(birthday)	1995-01-01 21:30:00
4	day10000 = timedelta(days=10000)	2022-05-19 21:30:00
5	someday = birthday + day10000	
6	print(someday)	

程式解說

- **第1 行**：從模組datetime匯入函式datetime與函式timedelta。
- **第2 行**：使用函式datetime 以「1995,1,1,21,30,0,0」為輸入，將此物件指定給變數birthday。

- **第3 行**：使用函式print 顯示變數birthday 到螢幕上。
- **第4 行**：使用函式timedelta 以「days=10000」為輸入，將此物件指定給變數day10000。

- **第5 行**：將變數birthday 加上變數day10000，將此物件指定給變數someday。
- **第6 行**：使用函式print 顯示變數someday到螢幕上。

11-3-4 使用模組time 計算程式執行時間

- 想要知道程式區塊執行所需時間，請在程式區塊前後加上模組time 的函式time
- 將結果指定給兩個變數，將後者減去前者就可以知道執行時間。

行號	範例 ( ：ch11\11-3-4- 模組 time 計算執行時間 .py)	執行結果
1	import time as t	執行所需時間爲 0.9958915710449219 秒
2	def count():	
3	st = t.time()	
4	[x for x in range(10000000)]	
5	et = t.time()	
6	print (' 執行所需時間爲 ',et-st,' 秒 ')	
7	count()	

程式解說

- **第1 行**：匯入模組time，重新命名為t。
- **第2 到6 行**：定義函式count，使用模組t的函式time，將此物件指定給變數st，執行串列生成式產生數字0 到9999999 的串列

- 使用模組t 的函式time將此物件指定給變數et，最後使用函式print 顯示「執行所需時間為」，et 減去st，加上「秒」。
- **第7 行**：呼叫函式count。

11-4 collections 套件

- 套件collections 包含模組OrderDict、deque 與Counter

11-4-1 模組OrderDict

重要函式與說明	程式碼與執行結果
collections.OrderedDict([items]) 使用 items 建立具有順序性的 OrderDict 物件。	import collections d = collections.OrderedDict([('Cr', 1),('To', 2)]) print(d)
	OrderedDict([('Cr', 1), ('To', 2)])

以下爲模組 OrderDict 範例，從範例中可以比較函式 dict 與函式 OrderDict 建立字典的差異。

行號	範例 ( ：ch11\11-4-1- 模組 OrderDict.py)	執行結果
1	import collections	{' 星期二 ': ' 跑步 ', ' 星期三 ': ' 籃球 ', ' 星期一 ': ' 游泳 ', ' 星期六 ': ' 棒球 ', ' 星期四 ': ' 桌球 ', ' 星期五 ': ' 羽球 ', ' 星期天 ': ' 休息 '}
2	days = [' 星期天 ', ' 星期一 ', ' 星期二 ', ' 星期三 ', ' 星期四 ', ' 星期五 ', ' 星期六 ']	
3	sport = [' 休息 ', ' 游泳 ',' 跑步 ', ' 籃球 ', ' 桌球 ', ' 羽球 ', ' 棒球 ']	
4	week = zip(days, sport)	OrderedDict([(' 星期天 ', ' 休息 '), (' 星期一 ', ' 游泳 '), (' 星期二 ', ' 跑步 '), (' 星期三 ', ' 籃球 '), (' 星期四 ', ' 桌球 '), (' 星期五 ', ' 羽球 '), (' 星期六 ', ' 棒球 ')])
5	d1 = dict(week)	
6	print(d1)	
7	week = zip(days, sport)	
8	d2 = collections.OrderedDict(week)	
9	print(d2)	

程式解說

- **第1 行**：匯入模組collections。
- **第2 行**： 設定days為串列「' 星期天', ' 星期一', ' 星期二', ' 星期三', ' 星期四', ' 星期五', ' 星期六' 」。

- **第3行：** 設定sport為串列「'休息', '游泳', '跑步', '籃球', '桌球', '羽球', '棒球'」。
- **第4 行：**使用函式zip 以串列days與串列sport 為輸入，將獲得的zip物件指定給變數week。
- **第5 行：**使用函式dict 以week建立字典，將獲得的字典物件指定給變數d1。

- **第6 行**：使用函式print 顯示變數d1 到螢幕上。
- **第7 行**：需要再次使用函式zip 以串列days 與串列sport 為輸入，將獲得的zip物件指定給變數week。

- **第8行**：使用模組collections的函式OrderedDict以變數week建立OrderedDict物件，將獲得的OrderedDict物件指定給變數d2。
- **第9 行**：使用函式print 顯示變數d2 到螢幕上。

11-4-2 模組deque

重要函式與說明	程式碼與執行結果
collections.deque(iterable) 使用 iterable 建立 deque 物件。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) print(dq)
	deque([1, 2, 3, 4, 5])
deque.pop() 從 deque 中刪除最右邊的元素。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) dq.pop() print(dq)
	deque([1, 2, 3, 4])

重要函式與說明	程式碼與執行結果
deque.popleft() 從 deque 中刪除最左邊的元素。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) dq.popleft() print(dq)
	deque([2, 3, 4, 5])
deque.append(x) 將 x 加到 deque 的右邊。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) dq.append(6) print(dq)
	deque([1, 2, 3, 4, 5, 6])

重要函式與說明	程式碼與執行結果
deque.appendleft(x) 將 x 加到 deque 的左邊。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) dq.appendleft(6) print(dq)
	deque([6, 1, 2, 3, 4, 5])
deque.remove(x) 從 deque 中第一個遇到數值爲 x 的元素刪除，若沒有發現數值爲 x 的元素，將發出 ValueError 錯誤。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) dq.remove(3) print(dq)
	deque([1, 2, 4, 5])

<table>
<tr><th>重要函式與說明</th><th>程式碼與執行結果</th></tr>
<tr><td rowspan="2">deque.count(x)
計算 deque 中數值 x 出現的次數。</td><td>from collections import deque
nums = [i for i in range(1,6)]
dq = deque(nums)
print(dq.count(3))</td></tr>
<tr><td>1</td></tr>
<tr><td rowspan="2">deque.rotate(n)
當 n 值大於 0，則 deque 中所有元素向右旋轉 n 個元素，超過部分依序補到 deque 的左邊；當 n 值小於 0，則 deque 中所有元素向左旋轉 n 個元素，超過部分依序補到 deque 的右邊。</td><td>from collections import deque
nums = [i for i in range(1,6)]
dq = deque(nums)
dq.rotate(1)
print(dq)</td></tr>
<tr><td>deque([5, 1, 2, 3, 4])</td></tr>
</table>

重要函式與說明	程式碼與執行結果
deque.reverse() 將 deque 所有元素反轉。	from collections import deque nums = [i for i in range(1,6)] dq = deque(nums) dq.reverse() print(dq)
	deque([5, 4, 3, 2, 1])

ch11\11-4-2- 模組deque.py

行號	範例 (：ch11\11-4-2- 模組 deque.py)	執行結果
1	from collections import deque	
2	nums = [i for i in range(1,6)]	
3	dq = deque(nums)	
4	print(dq)	
5	dq.rotate(1)	
6	print(dq)	deque([1, 2, 3, 4, 5])
7	dq.pop()	deque([5, 1, 2, 3, 4])
8	print(dq)	deque([5, 1, 2, 3])
9	dq.popleft()	deque([1, 2, 3])
10	print(dq)	deque([1, 2, 3, 8])
11	dq.append(8)	deque([8, 1, 2, 3, 8])
12	print(dq)	2
13	dq.appendleft(8)	deque([8, 3, 2, 1, 8])
14	print(dq)	
15	print(dq.count(8))	
16	dq.reverse()	
17	print(dq)	

程式解說

- **第1 行**：從套件collections匯入模組deque。
- **第2行**：設定nums為串列生成式「i for i in range(1,6)」，相當於產生數字1到5加到串列nums 中。

- **第3 行**：使用函式deque 以串列nums 為輸入，將獲得的物件deque 指定給物件dq。
- **第4 行**：使用函式print 顯示物件dq 到螢幕上。
- **第5 行**：使用物件dq 的函式rotate，以1 為輸入，讓物件dq 向右旋轉一個元素。

- **第6 行**：使用函式print 顯示物件dq 到螢幕上。
- **第7 行**：使用物件dq 的函式pop，取出物件dq 最右邊的元素。
- **第8 行**：使用函式print 顯示物件dq 到螢幕上。
- **第9 行**：使用物件dq 的函式popleft，取出物件dq 最左邊的元素。

- **第10 行**：使用函式print 顯示物件dq 到螢幕上。
- **第11行**：使用物件dq的函式append，以8為輸入，在物件dq最右邊新增數值8。
- **第12 行**：使用函式print 顯示物件dq 到螢幕上。
- **第13行**：使用物件dq的函式appendleft，以8為輸入，在物件dq最左邊新增數值8。

- **第14 行**：使用函式print 顯示物件dq 到螢幕上。
- **第15行**：使用函式print顯示物件dq的函式count，以8為輸入，顯示結果到螢幕上，
- 相當於顯示數值8 出現的次數。
- **第16 行**：使用物件dq 的函式reverse，反轉物件dq。
- **第17 行**：使用函式print 顯示物件dq 到螢幕上。

11-4-3 模組Counter

重要函式與說明	程式碼與執行結果
collections.Counter([iterable | mapping]) 可以使用 iterable 或 mapping 初始化 Counter 物件。	from collections import Counter x = ['a', 'a', 'b', 'b', 'b'] c = Counter(x) print(c) c = Counter({'a':2,'b':3}) print(c)
	Counter({'b': 3, 'a': 2}) Counter({'b': 3, 'a': 2})
Counter.elements() 將 Counter 物件依照元素與出現次數還原回原始串列，串列中每個元素都要符合出現頻率的次數。	from collections import Counter c = Counter({'a':2,'b':3,'c':1}) print(list(c.elements()))
	['a', 'a', 'b', 'b', 'b', 'c']

重要函式與說明	程式碼與執行結果
Counter.values() Counter 物件相當於字典，顯示 Counter 物件對應的出現次數。	from collections import Counter c = Counter({'a':2,'b':3,'c':1}) print(c.values())
	dict_values([2, 3, 1])
Counter.keys() Counter 物件相當於字典，顯示 Counter 物件對應的「鍵」。	from collections import Counter c = Counter({'a':2,'b':3,'c':1}) print(c.keys())
	dict_keys(['a', 'b', 'c'])
Counter.most_common([n]) 顯示 Counter 物件最常出現的前 n 個元素與出現次數，當沒有提供 n 值，則顯示所有元素與出現次數。	from collections import Counter c = Counter({'a':2,'b':3,'c':1}) print(c.most_common())
	[('b', 3), ('a', 2), ('c', 1)]

1. 模組Counter 初始化範例

行號	範例 (：ch11\11-4-3a- 模組 Counter 初始化 .py)	執行結果
1	from collections import Counter	
2	c = Counter('Python')	
3	print(c)	Counter({'y': 1, 't': 1, 'h': 1, 'n': 1, 'P': 1, 'o': 1})
4	x = ['a', 'a', 'b', 'b', 'b']	
5	c = Counter(x)	
6	print(c)	Counter({'b': 3, 'a': 2})
7	c = Counter({'a':2,'b':3})	
8	print(c)	Counter({'b': 3, 'a': 2})
9	c = Counter(a=2, b=3)	
10	print(c)	Counter({'b': 3, 'a': 2})

程式解說

- **第1 行**：從套件collections匯入模組Counter。
- **第2行**：將物件Counter以「Python」為輸入，將獲得的Counter物件指定給物件c。
- **第3 行**：使用函式print 顯示物件c 到螢幕上。
- **第4 行**：設定串列x 為「'a', 'a', 'b', 'b', 'b'」
- **第5 行**：將物件Counter 以串列x 為輸入，將獲得的Counter物件指定給物件c。

- **第6 行**：使用函式print 顯示物件c 到螢幕上。
- **第7行**：將物件Counter以「{'a':2,'b':3}」為輸入，將獲得的Counter物件指定給物件c。
- **第8 行**：使用函式print 顯示物件c 到螢幕上。
- **第9行**：將物件Counter以「a=2, b=3」為輸入，將獲得的Counter物件指定給物件c。
- **第10 行**：使用函式print 顯示物件c 到螢幕上。

2. 模組Counter 應用範例

計算出現最多次數與最少次數範例。

行號	範例 ( ：ch11\11-4-3b- 模組 Counter 的應用 .py)	執行結果
1	from collections import Counter	
2	c = Counter({'a':2,'b':3,'c':1})	Counter({'b': 3, 'a': 2, 'c': 1})
3	print(c)	['b', 'b', 'b', 'a', 'a', 'c']
4	print(list(c.elements()))	dict_values([3, 2, 1])
5	print(c.values())	dict_keys(['b', 'a', 'c'])
6	print(c.keys())	[('b', 3), ('a', 2), ('c', 1)]
7	print(c.most_common())	[('b', 3)]
8	print(c.most_common(1))	[('c', 1), ('a', 2), ('b', 3)]
9	print(c.most_common()[::-1])	[('c', 1)]
10	print(c.most_common()[:-2:-1])	

程式解說

- **第1 行**：從套件collections匯入模組Counter。
- **第2 行**：將物件Counter 以「{'a':2,'b':3,'c':1}」為輸入，將獲得的物件Counter 指定給物件c。
- **第3 行**：使用函式print 顯示物件c 到螢幕上。

- **第4 行**：使用函式list，以物件c 的函式elements 為輸入，最後使用函式print 將結果顯示到螢幕上。
- **第5行**：使用函式print顯示物件c函式values的結果到螢幕上，表示顯示物件c中所有元素的出現次數。
- **第6 行**：使用函式print 顯示物件c 函式keys 的結果到螢幕上，表示顯示物件c 中所有元素。

- **第7 行**：使用函式print 顯示物件c 的函式most_common，表示依照出現次數由高到低顯示物件c 所有元素與出現次數。
- **第8 行**：使用函式print 顯示物件c 的函式most_common，以1 為輸入，表示顯示物件c 中出現最多次數的元素與出現次數到螢幕上。

- **第9 行**：使用函式print 顯示物件c 的函式most_common，後方加上「[::-1]」，表示依照出現次數由低到高顯示物件c 所有元素與出現次數。
- **第10 行**：使用函式print 顯示物件c 的函式most_common，後方加上「[:-2:-1]」，表示顯示物件c 中出現最少次數的元素與出現次數到螢幕上。

模組Counter 的運算子

運算子	說明	程式碼與執行結果
a+b	a 與 b 的所有元素與出現次數進行累加。	from collections import Counter a = Counter(a=2, b=3) b = Counter(b=2, c=1) print(a + b)
		Counter({'b': 5, 'a': 2, 'c': 1})
a-b	a 的元素與次數，減去 b 的元素與次數。	from collections import Counter a = Counter(a=2, b=3) b = Counter(b=2, c=1) print(a - b)
		Counter({'a': 2, 'b': 1})

a&b	a 與 b 兩者都有的元素與次數，次數取較少者。	from collections import Counter a = Counter(a=2, b=3) b = Counter(b=2, c=1) print(a & b)
		Counter({'b': 2})
a|b	a 或 b 只要一個有就可以納入，次數取較多者。	from collections import Counter a = Counter(a=2, b=3) b = Counter(b=2, c=1) print(a | b)
		Counter({'b': 3, 'a': 2, 'c': 1})

3. 模組Counter 運算子範例

行號	範例 (：ch11\11-4-3c- 模組 Counter 的運算子 .py)	執行結果
1	from collections import Counter	Counter({'b': 5, 'a': 2, 'c': 1}) Counter({'a': 2, 'b': 1}) Counter({'b': 2}) Counter({'b': 3, 'a': 2, 'c': 1})
2	a = Counter(a=2, b=3)	
3	b = Counter(b=2, c=1)	
4	print(a + b)	
5	print(a - b)	
6	print(a & b)	
7	print(a | b)	

程式解說

- **第1 行**：從套件collections匯入模組Counter。
- **第2行**：將物件Counter以「a=2, b=3」為輸入，將獲得的Counter物件指定給物件a。
- **第3行**：將物件Counter以「b=2, c=1」為輸入，將獲得的Counter物件指定給物件b。

- **第4 行**：使用函式print 顯示「a + b」運算結果到螢幕上。
- **第5 行**：使用函式print 顯示「a - b」運算結果到螢幕上。
- **第6 行**：使用函式print 顯示「a & b」運算結果到螢幕上。
- **第7 行**：使用函式print 顯示「a | b」運算結果到螢幕上。

11-4-4 找出文字檔中出現次數最多的5 個字

行號	範例 ( ：ch11\11-4-4- 找出出現次數最多的 5 個字 .py)	執行結果
1	from collections import Counter	
2	import re	
3	fin = open('zen.txt', 'rt')	[('is', 10), ('better', 8),
4	s = fin.read().lower()	('than', 8), ('the', 6),
5	words = re.findall(r'[\w\']+',s)	('to', 5)]
6	c = Counter(words)	
7	print(c.most_common(5))	

程式解說

- **第1 行**：從套件collections匯入模組Counter。
- **第2 行**：匯入模組re。
- **第3 行**：開啟檔案「zen.txt」，以純文字與讀取模式開啟檔案，將回傳的物件指定給物件fin。

- **第4行**：物件fin使用函式read，一次讀取檔案內所有內容，在使用函式lower轉換成小寫字母，將結果指定給物件s。
- **第5行**：使用模組re的函式findall，找出由大小寫英文字母、數字、底線與單引號(')所組成的單字，越長越好，將結果指定給物件words。

- **第6 行**：將物件Counter 以words 為輸入，將獲得的Counter物件指定給物件c。
- **第7 行**：使用函式print 顯示物件c 的函式most_common，以5 為輸入，表示顯示物件c 中出現次數前5 多的元素與出現次數到螢幕上。

11-5 綜合應用—備份資料夾

行號	範例 ( ：ch11\11-5- 備份資料夾 .py)
1	import os
2	import time
3	src_dir = ['d:\\blog']
4	for d in src_dir:
5	index=d.rfind('\\')
6	fmt = '%Y%m%d_%H%M%S'
7	target_file = 'd:\\backup\\' + d[index+1:] + '_' + time.strftime(fmt) + '.zip '
8	zipcmd = '7z a -tzip ' + target_file + d
9	if os.system(zipcmd) == 0:
10	print(' 備份成功 ')
11	else:
12	print(' 備份失敗 ')

程式解說

- **第1 行**：匯入模組os。
- **第2 行**：匯入模組time。
- **第3 行**：設定串列src_dir 為「'd:\\blog'」，表示需要備份的資料夾，可以加入多個資料夾，加入串列src_dir 的資料夾都會進行備份。

- **第4 到12 行**：使用迴圈取出串列src_dir 的每個資料夾到字串物件d，找出字串物件d 從右邊找過來的第一個字元「\」的索引值指定給物件index(第5 行)
- 設定時間格式fmt 為「%Y%m%d_%H%M%S」(第6 行)

- 設定字串物件target_file(備份檔案所在路徑) 為「d:\\backup\\」，串接字串物件d的index 的下一個索引值開始到最後，也就是備份的資料夾名稱，串接目前時間的日期與時間(time.strftime(fmt))，最後串接「.zip」(第7 行)

- 設定字串物件zipcmd(壓縮資料夾指令)為字串「7a -tzip」，表示使用7將資料壓縮為zip 格式的壓縮檔，串接字串物件target_file，串接字串變數d(第8 行)。

- 若使用模組os 的函式system 執行壓縮資料夾指令zipcmd，如果執行成功，則顯示「備份成功」，否則顯示「備份失敗」(第9 到12 行)。